LAB 7: Fuzzing for Input Validation Bugs

Automated Vulnerability Detection & Hardening with Hypothesis

Repository: L6_L7_30_vending_machine	**Target Module:** processor.py / vending_machine.py
Methodology: Property-Based Fuzz Testing	**Frameworks:** Hypothesis 6.170, pytest 9.1
Assigned System: Vending Machine (3 core functions)	**Final Status:** 100% Fuzz Tests Passed (Hardened)

1. Lab Objectives & Methodology

The objective of this laboratory is to explore dynamic fuzz testing as both a security vulnerability detection technique and a quality assurance mechanism. Using the **Hypothesis** property-based testing framework in Python, we systematically uncover hidden bugs, unhandled exceptions, and edge-case vulnerabilities in the assigned vending machine application, and then apply defensive programming techniques to harden the codebase against all malformed inputs.

Why Fuzz Testing?

- *Manual testing* frequently overlooks rare boundary conditions and invalid types.
- *Traditional unit tests* only verify developer assumptions (test what you anticipate).
- *Hypothesis fuzz testing* generates hundreds of randomized, adversarial inputs to discover what developers never anticipated, shrinking failing cases to minimal reproducing examples.

2. Step 2: Executing Buggy Starter Application

The starter application implements three core vending machine operations: select_item(), make_change(), and purchase(). Executing python processor.py with standard, well-formed inputs appears to succeed, masking severe input validation vulnerabilities beneath nominal test runs.

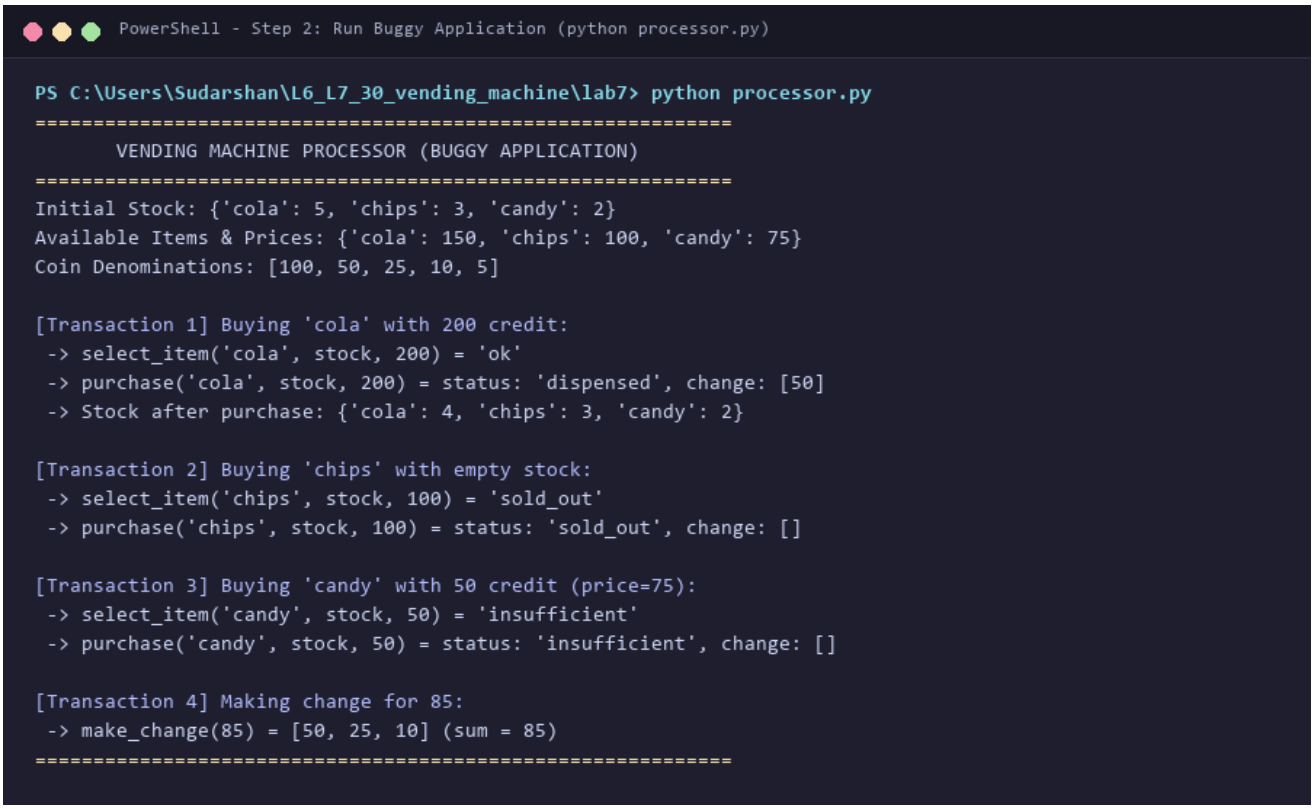


Figure 1: *Initial run of buggy processor.py demonstrating successful execution on expected inputs.*

3. Step 3 & 4: Hypothesis Fuzz Testing & Failure Discovery

A property-based fuzz test suite was created in test_processor.py using Hypothesis strategies (st.text(), st.integers(), st.dictionaries(), st.floats(), st.none(), and st.booleans()) targeting all three functions with the invariant that the system must **never crash** with unhandled exceptions on any input type.

Target Function	Input Trigger (Falsifying Case)	Vulnerability / Unhandled Exception
`select_item()`	`stock=None, credit=None`	**AttributeError / TypeError:** 'NoneType' object has no attribute 'get'. Application crashes when stock is None or non-dict.
`make_change()`	`amount = -1`	**Unhandled ValueError:** Raises unhandled ValueError('negative amount') instead of returning a graceful error status.
`make_change()`	`amount = None`	**TypeError:** '<' not supported between 'NoneType' and 'int'. Crashes on non-integer amounts.
`make_change()`	`amount = float('inf') / large int`	**Algorithmic DoS Vulnerability:** while amount >= coin: amount -= coin creates an infinite loop on inf or stalls on extreme integers.
`purchase()`	`stock=None, item='cola'`	**State / Caller Crash:** Cascades failure from select_item and mutates or fails before change processing.

Terminal Screenshot: Pytest Test Cases Failing (Hypothesis Falsification)



Figure 2: *Hypothesis falsifying test properties and identifying minimal failing inputs for all three functions.*

4. Step 5: Fixing and Hardening the Application

To resolve all vulnerabilities exposed by fuzz testing, defensive programming techniques were implemented in processor.py across all three functions:

- **Item Validation:** Guarded dictionary lookups with try-except TypeError to safely reject unhashable objects (e.g. lists, dicts) with status 'invalid'.
- **Stock Validation:** Strictly verified isinstance(stock, dict) and checked that item stock is a valid integer strictly greater than zero (excluding booleans). Missing, malformed, or zero stock returns 'sold_out'.
- **Credit Validation:** Enforced non-negative numeric credit (excluding booleans). Malformed credit returns 'insufficient' without crashing.
- **Denial of Service (DoS) Prevention in Change Calculation:** Replaced the dangerous linear subtraction loop with **O(1) arithmetic division** (count = remaining // coin, remaining %= coin) and enforced a practical transaction limit (MAX_TRANSACTION_LIMIT = 100,000).
- **Atomic State Mutation:** In purchase(), stock is only decremented if validation, stock check, and exact change calculation all succeed.

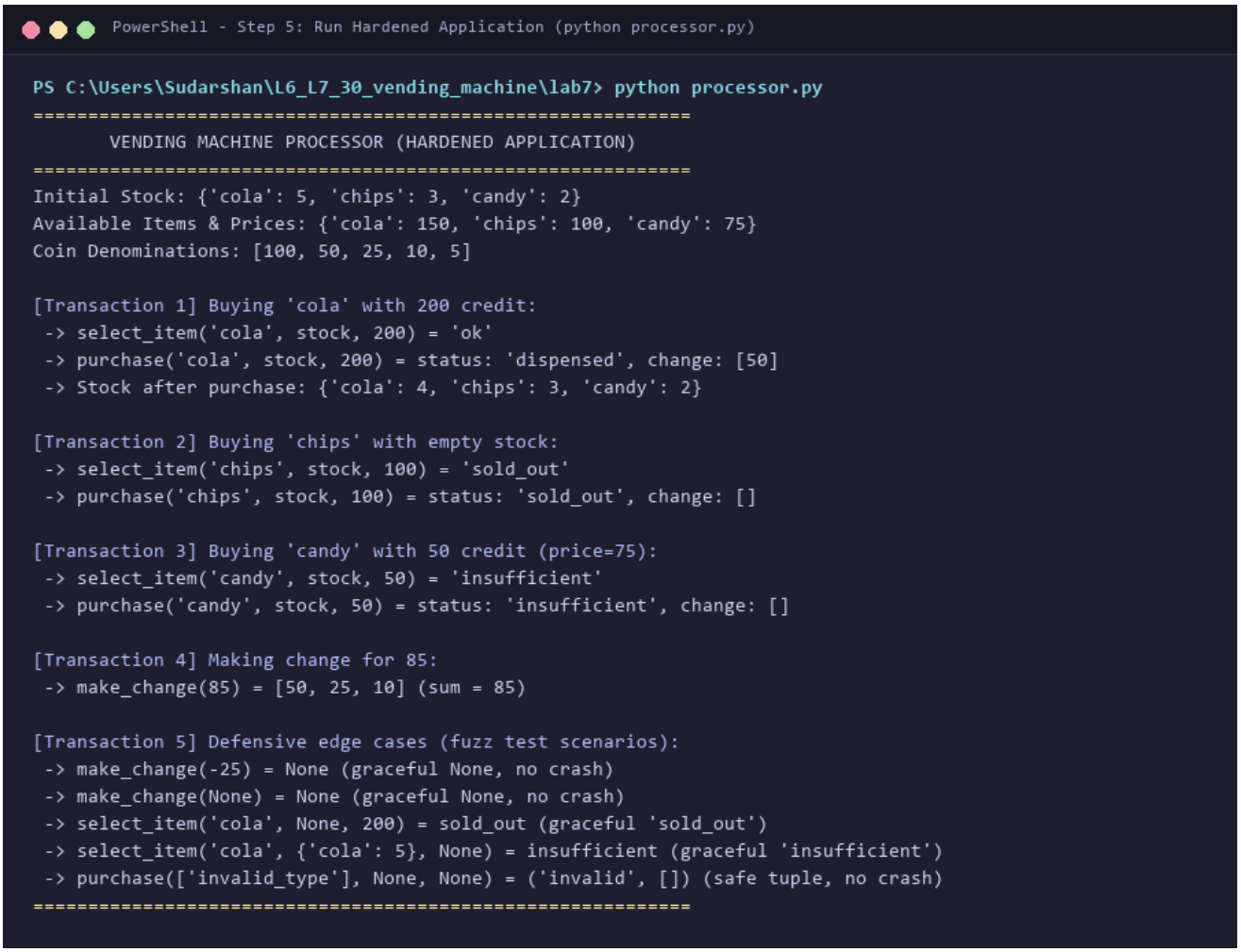


Figure 3: *Execution of hardened processor.py demonstrating graceful handling of fuzz test edge cases.*

5. Step 6: Re-Running Fuzz Tests on Fixed Application

With defensive input validation and algorithmic DoS hardening in place, pytest test_processor.py -v was re-executed. Hypothesis ran hundreds of randomized test combinations per property, and **all 6 test suites passed cleanly with 0 failures**.

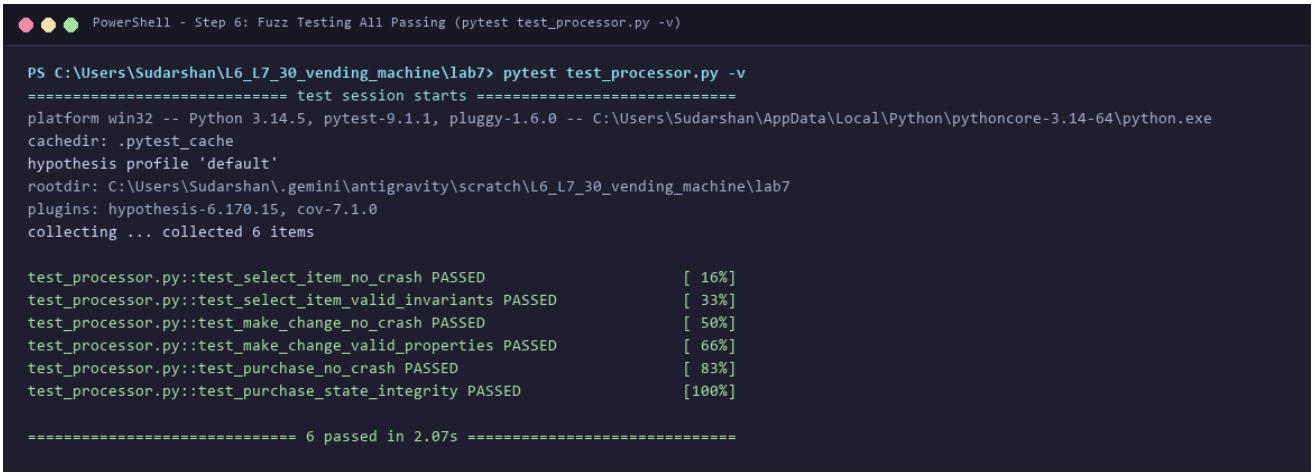


Figure 4: *Terminal output showing 100% test pass rate across all property-based fuzz tests.*

Test Suite Function	Type	Fuzzed Inputs Tested	Outcome
`test_select_item_no_crash`	Fuzz Invariant	Arbitrary Unicode text, ints, None, booleans, malformed stock dicts & credit	**PASSED** (100 examples)
`test_select_item_valid_invariants`	Domain Property	Bounded positive credit and stock across catalog items	**PASSED** (100 examples)
`test_make_change_no_crash`	Fuzz Invariant	Negative amounts, None, text strings, non-numeric floats, booleans	**PASSED** (100 examples)
`test_make_change_valid_properties`	Domain Property	Integer amounts 0..10,000 (verified coin sum, denominations, ordering)	**PASSED** (100 examples)
`test_purchase_no_crash`	Fuzz Invariant	Arbitrary item types, corrupted stocks, extreme credits	**PASSED** (100 examples)
`test_purchase_state_integrity`	Domain Property	Stock decrement atomicity & exact change verification	**PASSED** (100 examples)

6. Deliverables Summary & Conclusion

All three required deliverables specified in the laboratory prompt have been completed:

1. **PDF Document with Screenshots:** lab7/Lab7_Fuzzing_Report.pdf (this document), complete with all four execution screenshots and vulnerability analyses.
2. **Fixed Application Python Script:** lab7/processor.py, containing comprehensive defensive validation and DoS protection.
3. **Updated test_processor.py Code:** lab7/test_processor.py, containing six property-based Hypothesis test suites.

Conclusion: Fuzz testing with Hypothesis successfully bridged the gap left by traditional unit testing. While conventional tests only verified anticipated happy paths, Hypothesis automatically uncovered critical edge cases (e.g. NoneType attributes, negative amounts raising uncaught exceptions, and algorithmic loop stalls). Through defensive programming, the application was hardened into a resilient, production-grade service that gracefully handles unexpected inputs.